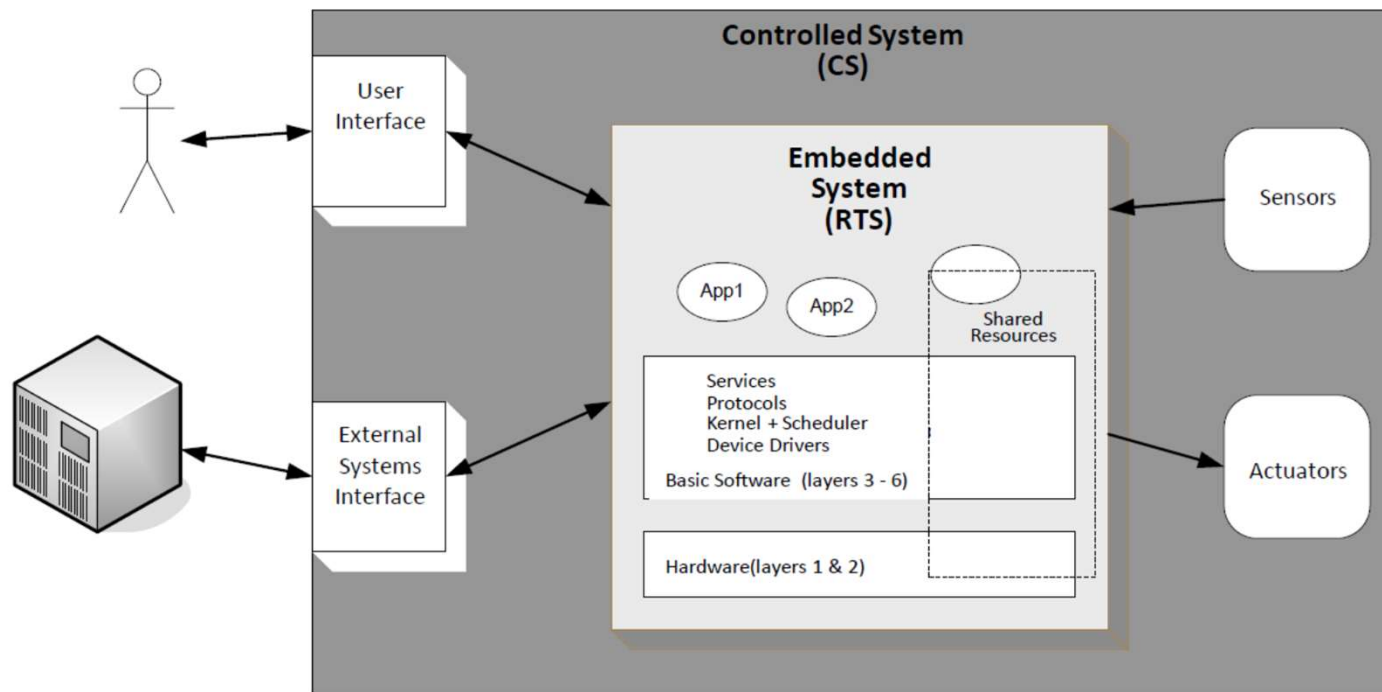


9- MODELLING REACTIVE SYSTEMS WITH STATECHARTS

REACTIVE SYSTEMS

- React to external events
as opposed to **transformational** systems
- May generate internal events and react to them too
- react means:
changes state, changes internal variables, execute actions,
produces output

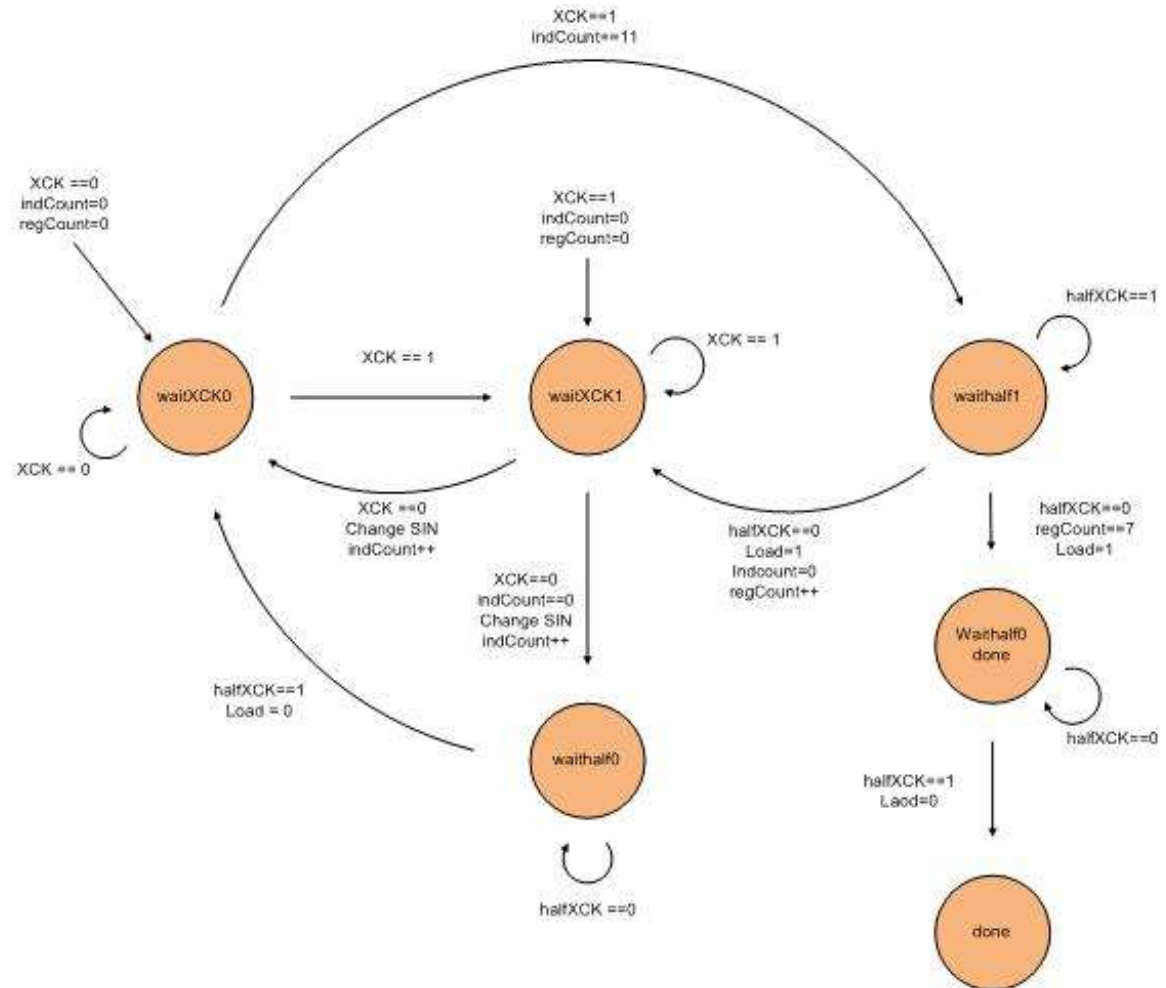
GENERIC STRUCTURE OF AN EMBEDDED SYSTEM



based on: Renaux, D. and Stadzisz, P. , Software Embarcado. In: SBC – Escola Regional de Informática (ERI), 2007

FINITE STATE MACHINE

- long history



FINITE STATE MACHINE

- Modelling tool
- Represented by a State Machine Diagram
- Adequate for modelling of the dynamic (behavioral) aspects of a reactive system/artifact
(reactive vs transformational)
- **Finite** State Machine (FSM) - number of states is finite
- States vs Transition
transitions are usually in reaction to external events

FINITE STATE MACHINES

Can be used to model behavior at several abstraction levels:

- Classes
- Use Cases
- Components
- Sub-systems
- Systems

STATECHARTS

David Harel (Weizmann Institute) - Israel, 1984

Contributions to FSM:

- Hierarchy
- History
- Concurrency

UML

Unified Modeling Language

- Originally by Grady Booch, James Rumbaugh and Ivar Jacobson
- Based on the integration of several existing modeling languages
- OMG standard (www.omg.org) in 1998
- Language based on visual models
- Current version: 2.5.1 (Dec 2017): omg.org/spec/UML
- Non-proprietary language

MODELING

The basic elements of a structural model are:

- THINGS
- Interaction among THINGS

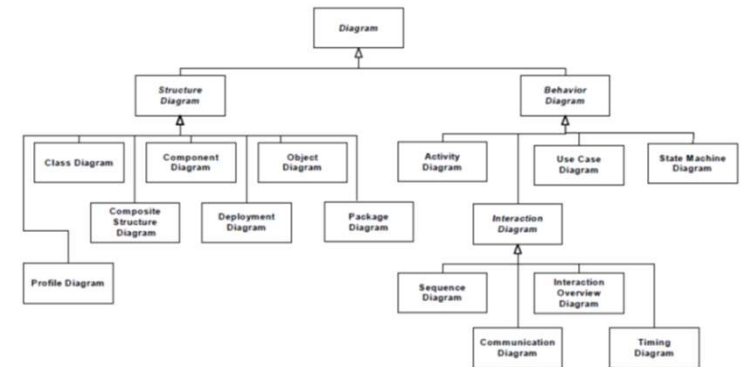


Figure A.5 The taxonomy of structure and behavior diagrams

THINGS

Physical things:

- coffee bean, processor, equipment , vehicle, planet ...

Logical things:

- bank account, contract, variable stored in memory ...

INTERACTION AMONG THINGS

Examples:

processor is connected to memory

personA and personB are married

personX is responsible for bank_account_12345

STATECHARTS IN UML 2.X

Two types of FSM

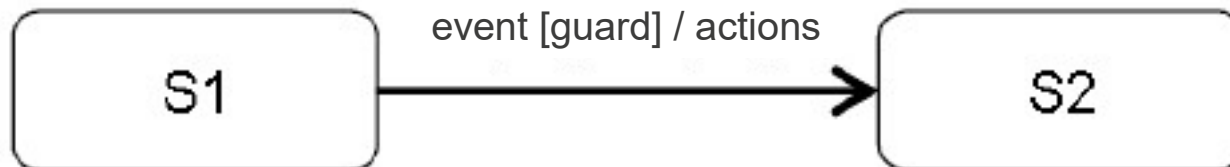
- **Behavioral**: describe the behavior of a reactive system
 - Protocol: FSM that describe the behavior of a classifier (no reference to implementation, just to the interface)
-
- References:
 - UML standard by OMG: <https://www.omg.org/spec/UML/2.5.1/PDF>
 - State Diagrams Tutorial: <https://warren2lynch.medium.com/state-diagram-comprehensive-guide-with-examples-e08b6d1c70fe>

STATECHARTS MODELING CONCEPTS

- **State**
represent a **stable condition/situation** of the system.
Named by a noun or present participle.
- **Event**
Relevant event occurring “instantly”
External vs Internal
- **Transition**
Changes from one state to another (or the same).
Transitions may be caused by: event, condition, end of activity, ...

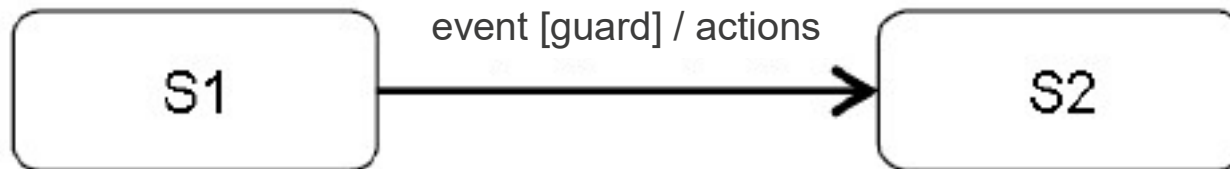
NOTATION

- state - S1 and S2 - names such as 'Idle', 'Locked'
- transition - arrow from current state to next state
rem: current and next state may be the same
- event - identifier of an event
- guard - logic expression (thus, results in true or false)
- actions - list of actions separated by semi-colon. Instantaneous.



SEMANTICS

- trigger: what cause the transition to occur
 - external event
 - internal event
 - time event
 - guard becoming true
 - end of activity in a state
- semantics: current state is S1, when an 'event' occurs, the guard condition is evaluated, if true, actions are executed in sequence (RTC). Then, the current state changes to S2.



event [guard] / actions
all three fields are optional

GRAPHICAL NOTATION

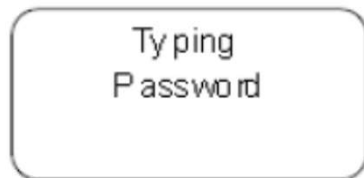


Figure 14.4 State notation



Figure 14.5 State with a name tab

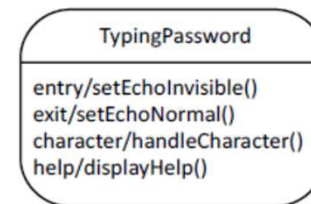
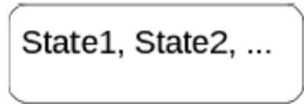


Figure 14.6 State with compartments

GRAPHICAL NOTATION

	<code>UMLStateShape (State)</code> <code>- UMLNameLabel (State)</code>
	<code>UMLStateShape (State+)</code> <code>- UMLNameLabel (State+)</code>
	<code>UMLStateShape (State,</code> <code>modelElement.isComposite=true,</code> <code>diagram.isCollapseStateIcon=true)</code> <code>- UMLNameLabel (State)</code>

GRAPHICAL NOTATION



UMLStateShape (State)
 - UMLNameLabel (State)
 - UMLLabel (Constraint)
 - UMLKeywordLabel (State)
 - UMLCompartment
 --- UMLStateShape (State, owned)
 ----- UMLNameLabel (State)
 - UMLCompartment
 --- UMLLabel (Behavior)
 - UMLCompartment
 --- UMLLabel (Transition)



UMLStateShape (State, diagram.
 diagram.isInheritedLighter=false,
 diagram.isInheritedDashed=true)
 - UMLNameLabel (State)



UMLStateShape (State,
 diagram.isInheritedLighter=true,
 diagram.isInheritedDashed=false)
 - UMLNameLabel (State)

STATECHARTS VS ACTIVITY DIAGRAMS

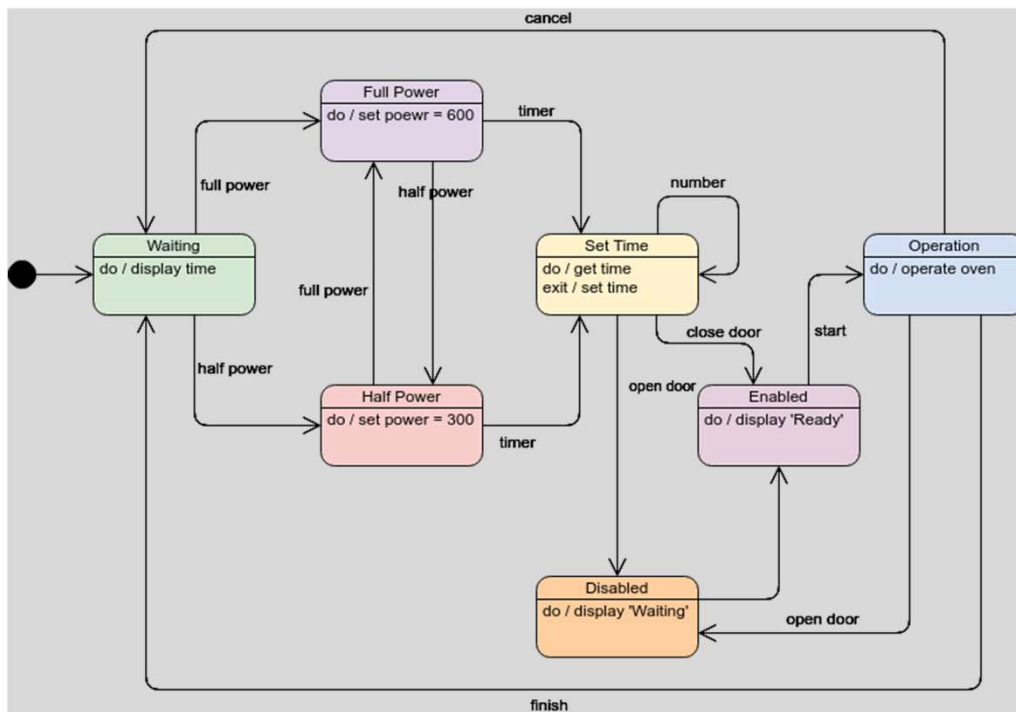
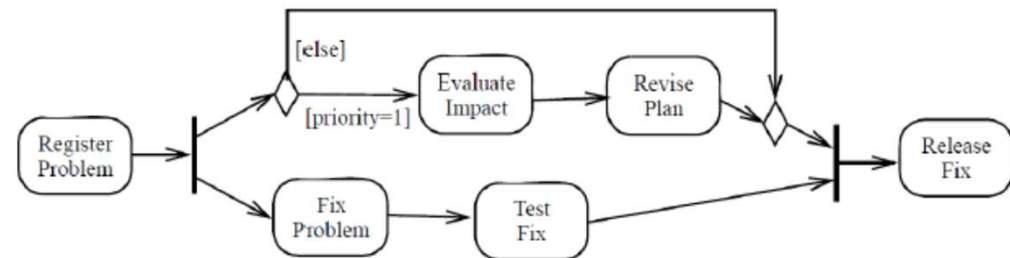


Figure 15.19 ActivityEdge connector example



STATES ACTIONS AND BEHAVIOR

- 'entry' - list of actions to perform when entering this state
- 'exit' - list of actions to perform when exiting this state
- 'event name' - list of actions to perform when this event occurs
- Rem: no state change, no exit / reentry
- 'do' - name of activity to perform while in this state

Activity - has duration:

- limited
- unlimited

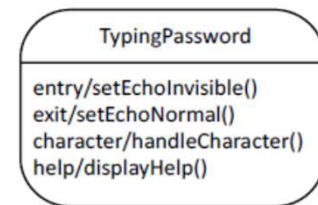


Figure 14.6 State with compartments

PSEUDO STATES

They are the origin or detination of transitions, thus, graphically behave as states

No duration associated to these pseudo-state.

PSEUDO-STATES: GRAPHICAL NOTATION

	UMLShape (Pseudostate, initial)
	UMLShape (FinalState)
	UMLShape (Pseudostate, shallowHistory)
	UMLShape (Pseudostate, deepHistory)
	UMLShape (Pseudostate, entryPoint)
	UMLShape (Pseudostate, exitPoint)
	UMLShape (Pseudostate, junction)
	UMLShape (Pseudostate, choice)
	UMLShape (Pseudostate, terminate)

	UMLStateShape (State) - UMLNameLabel (State) - UMLShape (ConnectionPointReference, entry point) --- UMLNameLabel (ConnectionPointReference)
	UMLStateShape (State) - UMLNameLabel (State) - UMLShape (ConnectionPointReference, exit point) --- UMLNameLabel (ConnectionPointReference)
	UMLShape (Pseudostate, join/fork)

STATECHART - EXAMPLES

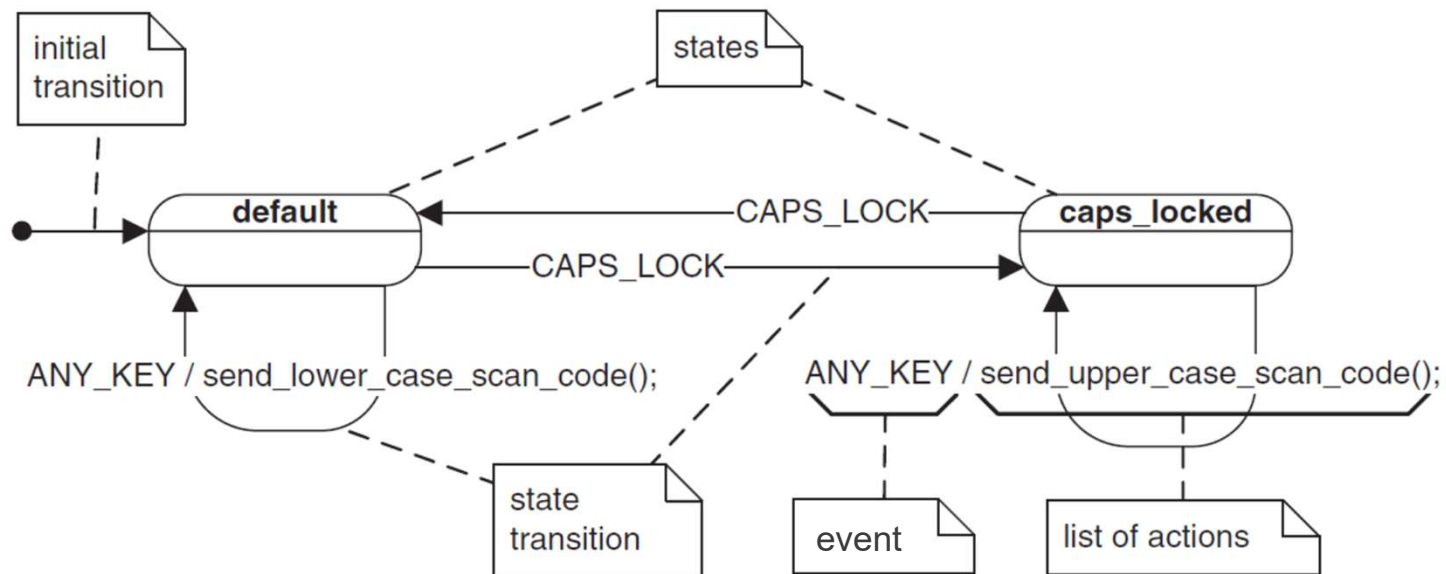
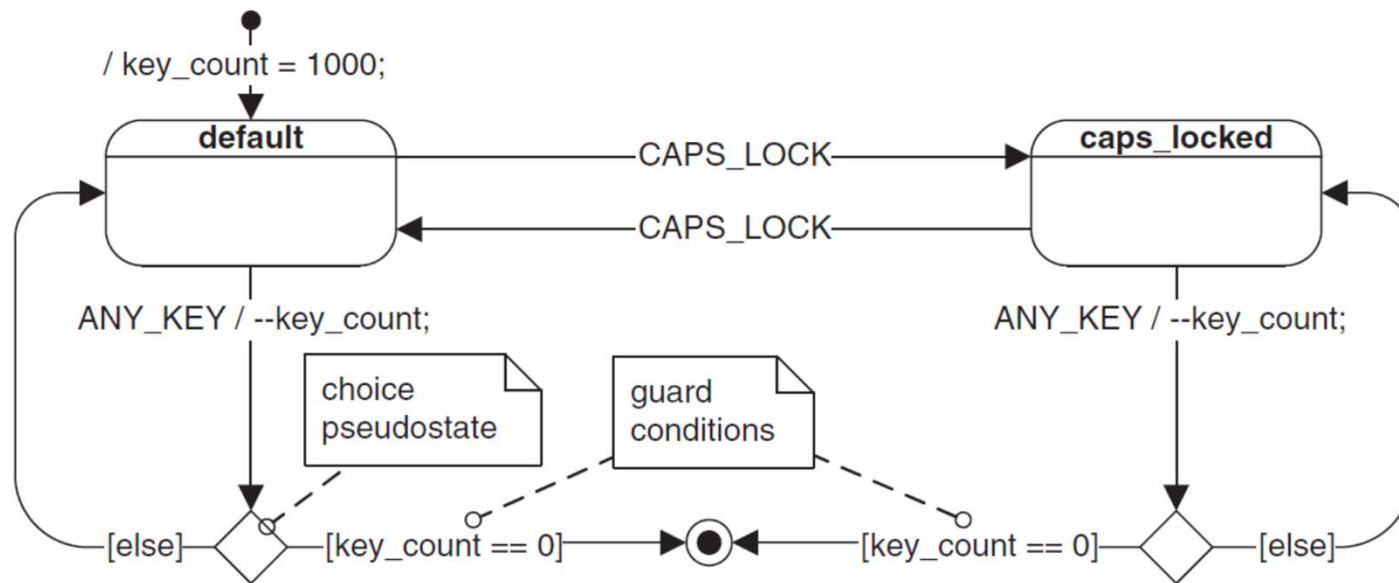


Figure 2.2: UML state diagram representing the computer keyboard state machine.

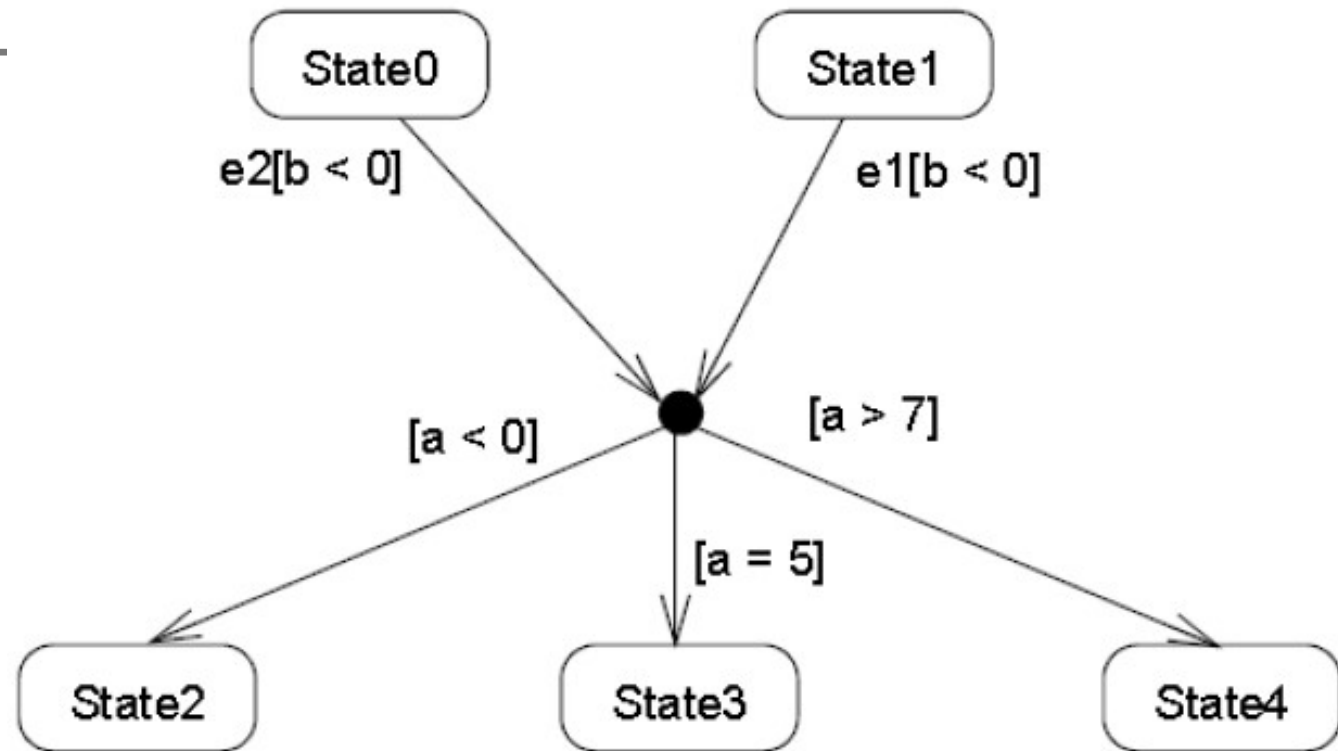
STATECHART - EXAMPLES



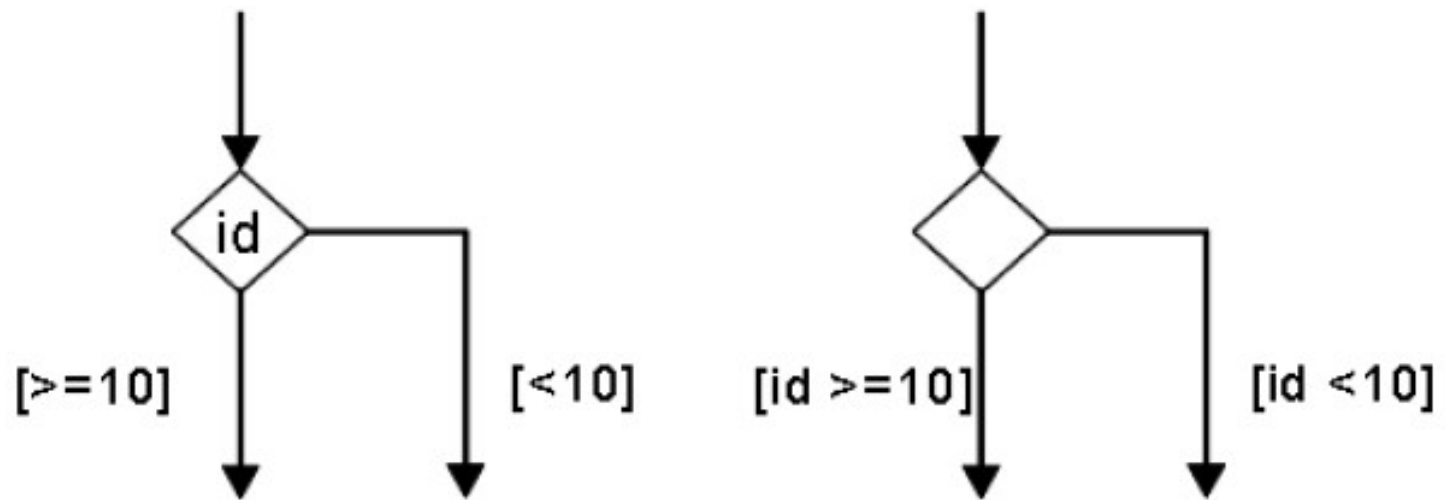
'else' is a keyword

Figure 2.4: Extended state machine of “cheap keyboard” with extended state variable `key_count` and various guard conditions.

JUNCTION

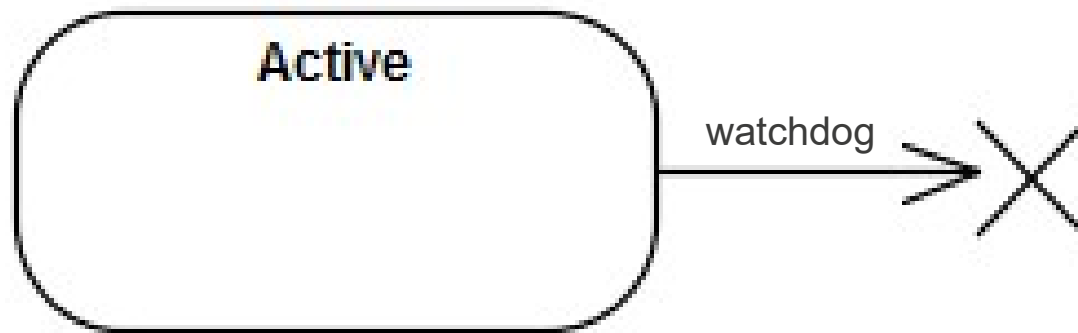


CHOICE



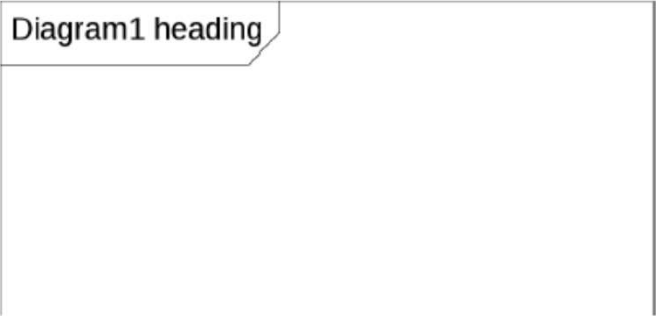
TERMINATE

- Equivalent to a call to `exit()` in an application



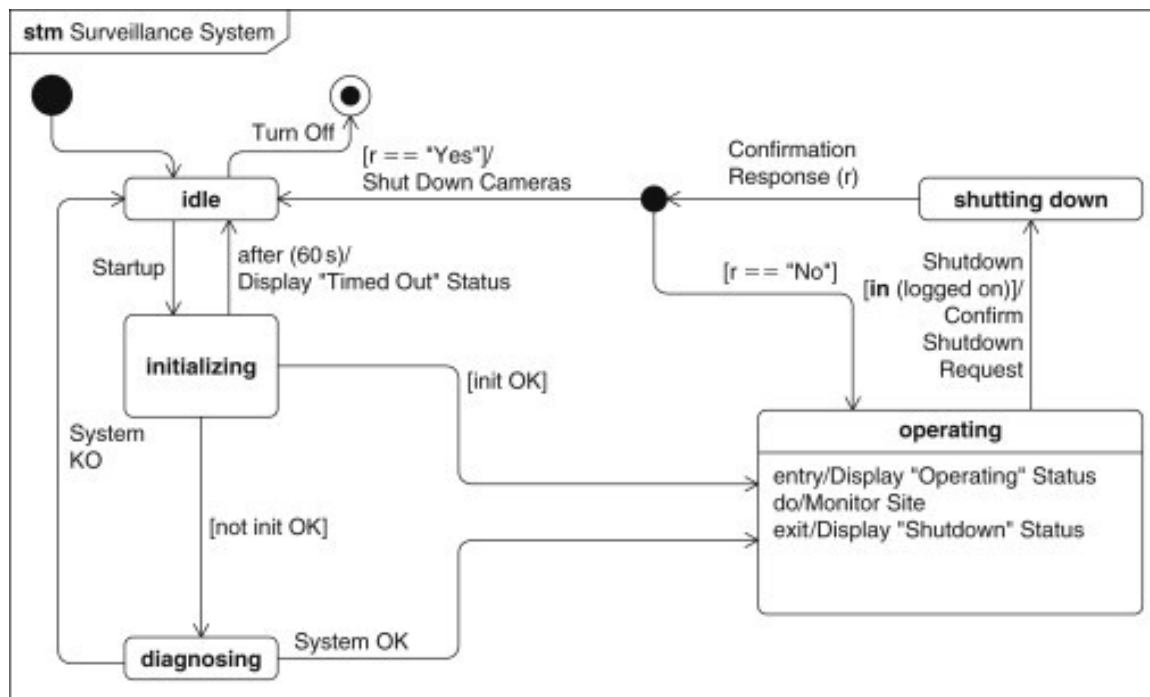
GRAPHICAL NOTATION - NON-SEMANTIC ELEMENTS

- frame and heading
- comment

Notation	Diagram elements
 Diagram1 heading	UMLDiagram (Element, isFrame=true) - UMLLabel (Element)
 Comment1 body	UMLShape - UMLLabel (Comment) UMLEdge

TIME EVENTS

- 'at' (absolute time expression)
event occurs at specified time
- 'after' (relative time expression)
event occurs RT after last entering that state



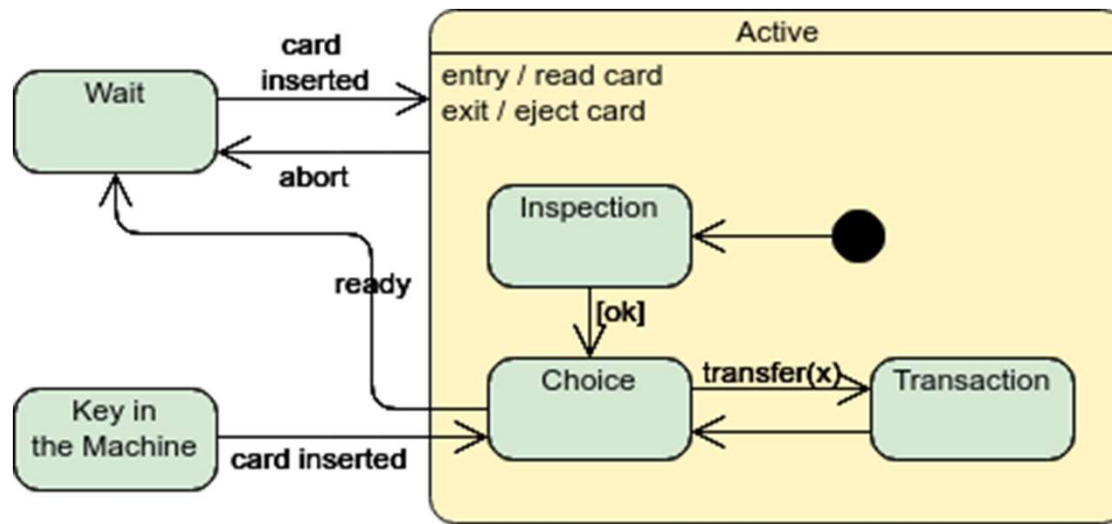
keywords:
'in'
'after'

HIERARCHY IN UML STATECHARTS

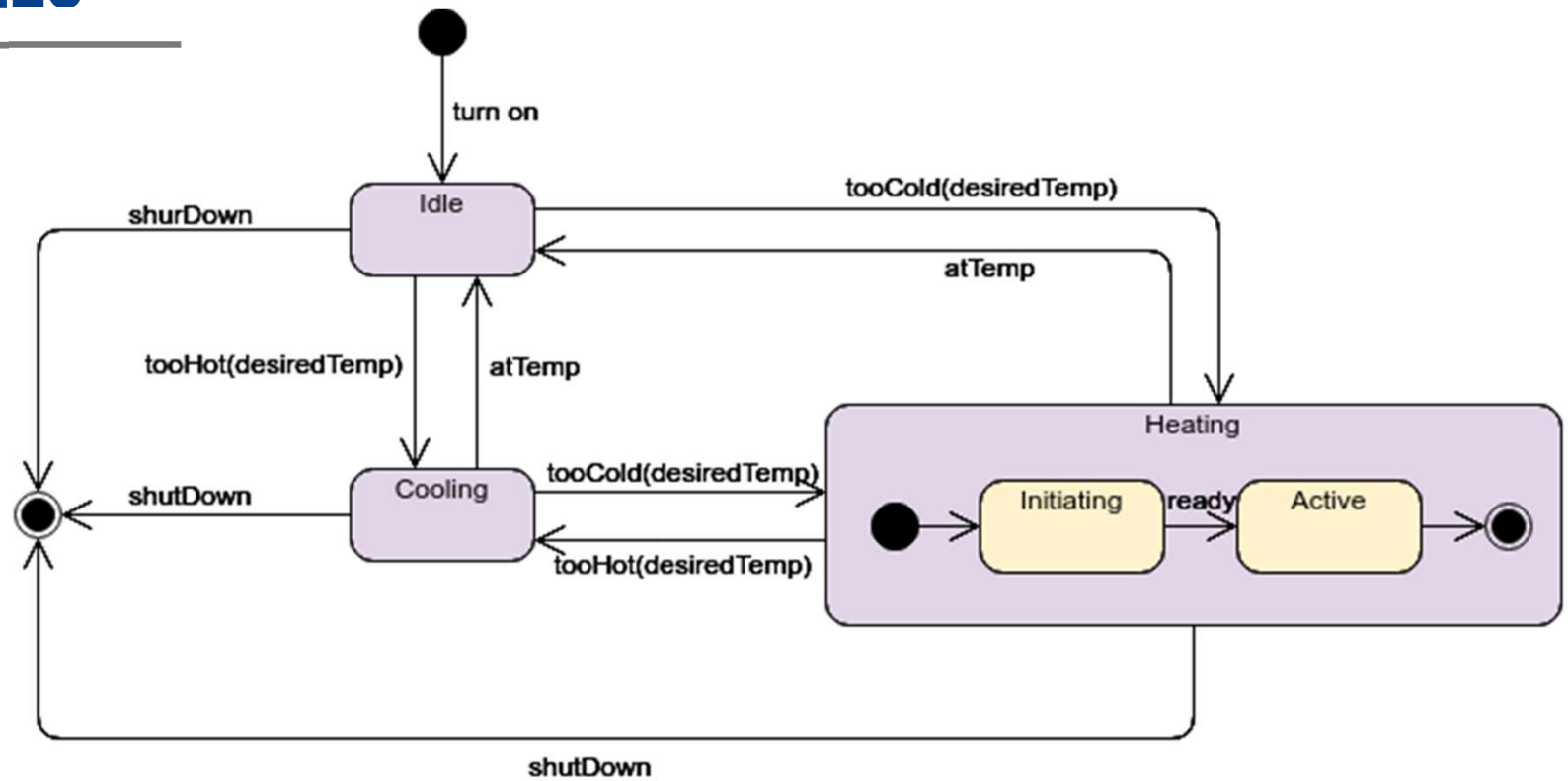
OR superstate

- can be in only sub-state of the superstate
- typically has an initial state
- may have a final state
- entering transition:
 - border
 - to internal state
- exit transition: from border or from internal state

EXAMPLES



EXAMPLES



HIERARCHY IN STATECHARTS

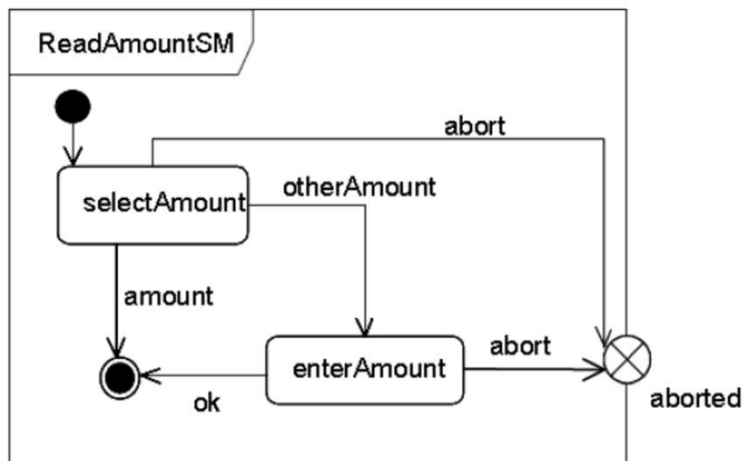


Figure 14.13 StateMachine with an exit point on the border

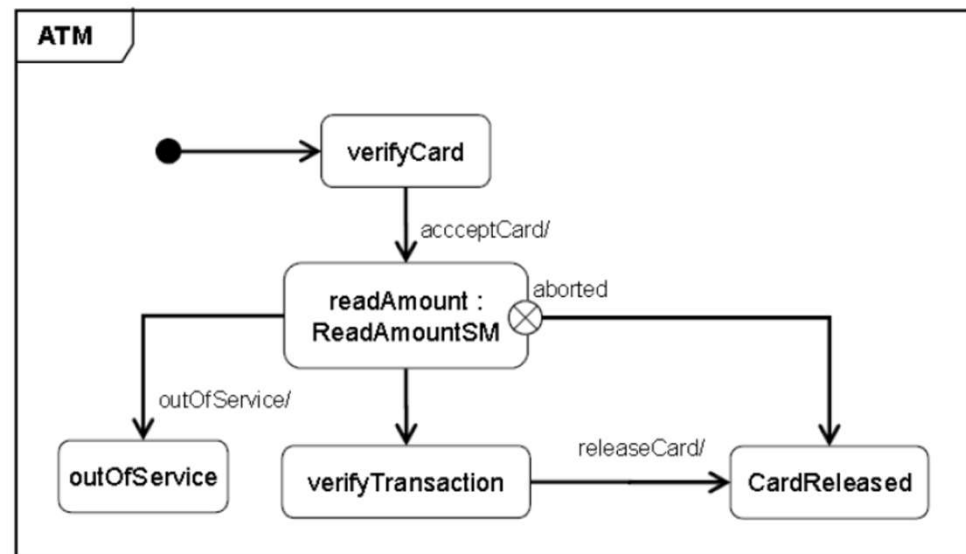


Figure 14.14 Submachine State that uses an exit point

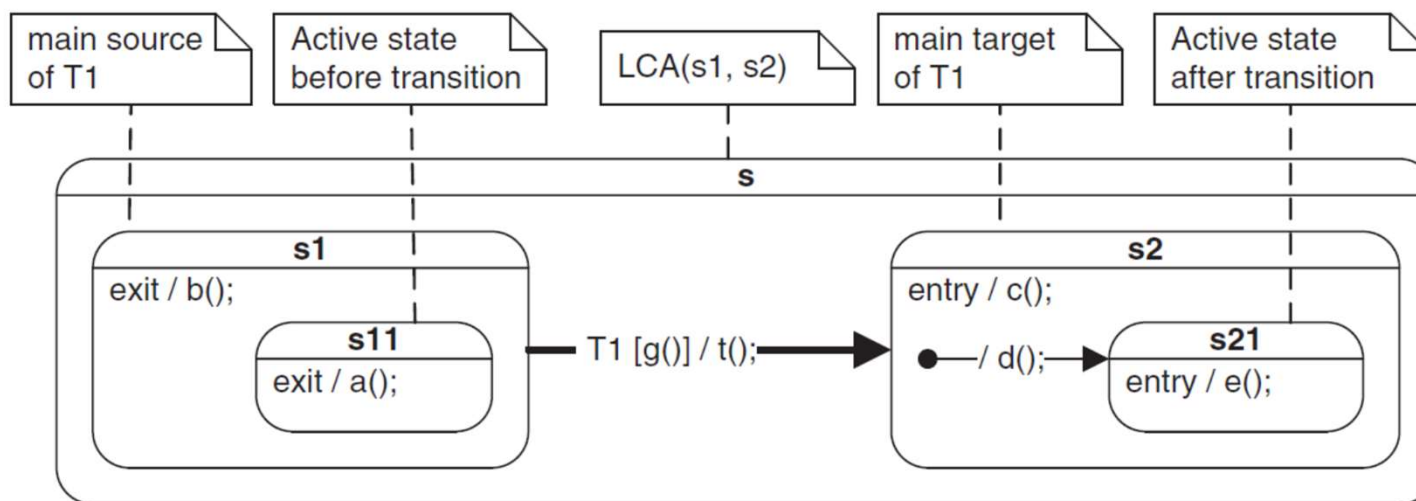


Figure 2.9: State roles in a state transition.

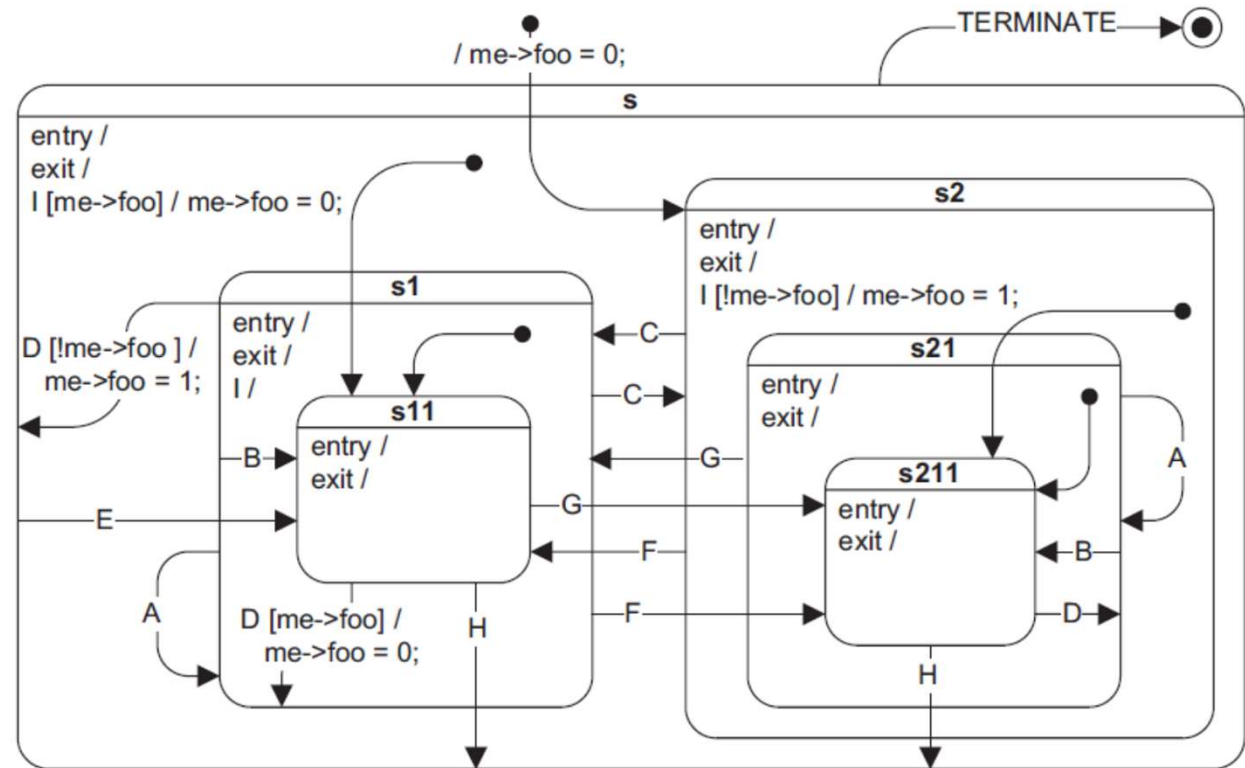


Figure 2.11: Hypothetical state machine that contains all possible state transition topologies up to four levels of state nesting.

HIERARCHY IN STATECHARTS - NOTATION

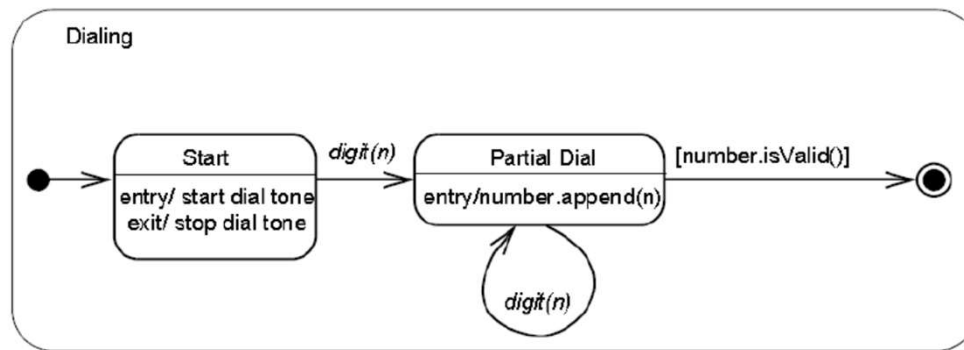


Figure 14.7 Composite State with two States

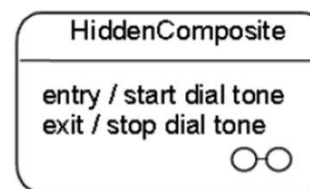


Figure 14.8 Composite State with a hidden decomposition indicator icon

HIERARCHY IN STATECHARTS

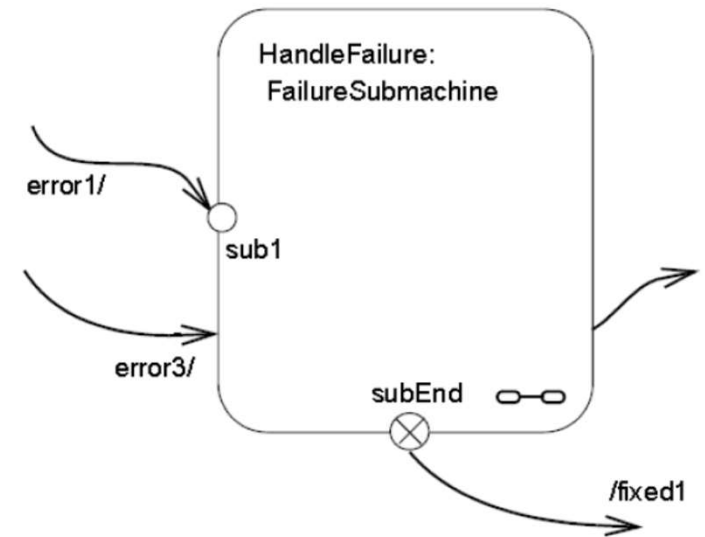
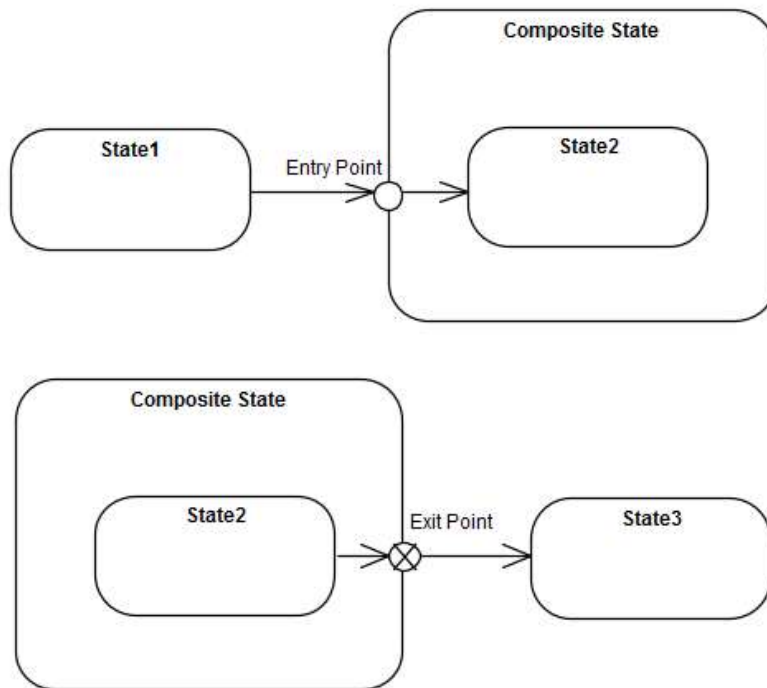
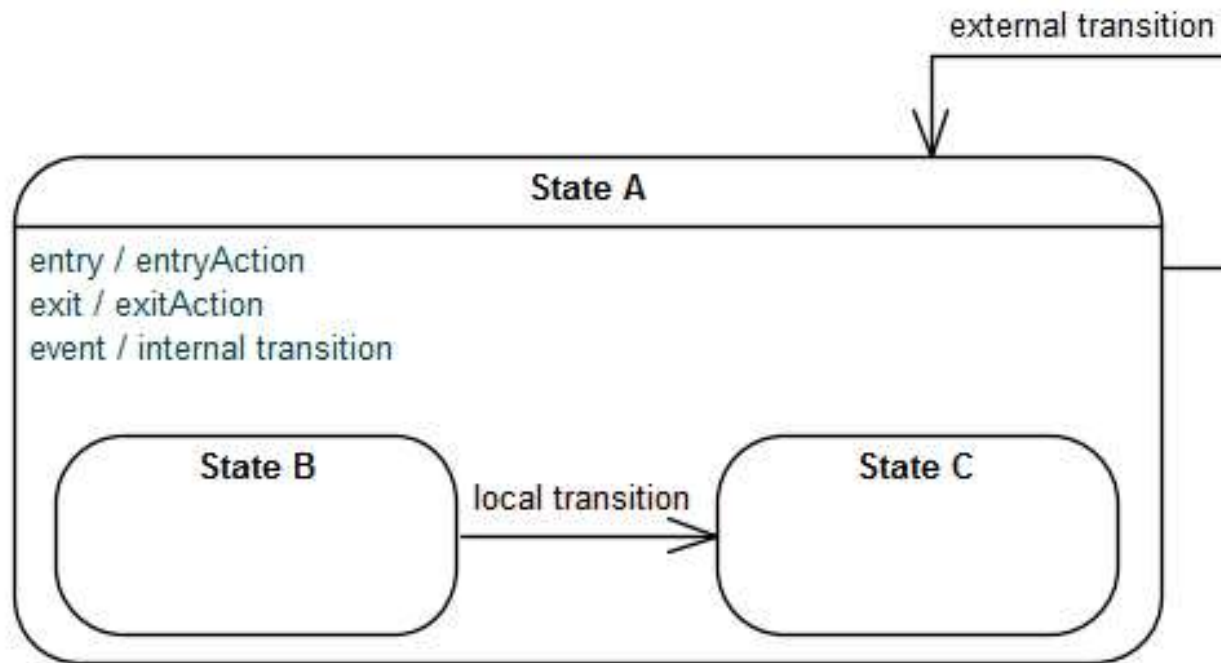


Figure 14.11 Submachine State example

TRANSITIONS - INTERNAL VS LOCAL VS EXTERNAL



TRANSITIONS - LOCAL VS EXTERNAL

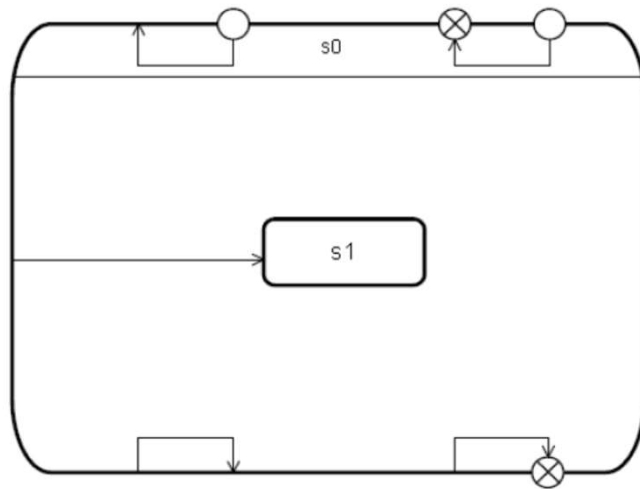


Figure 14.34 Local Transitions

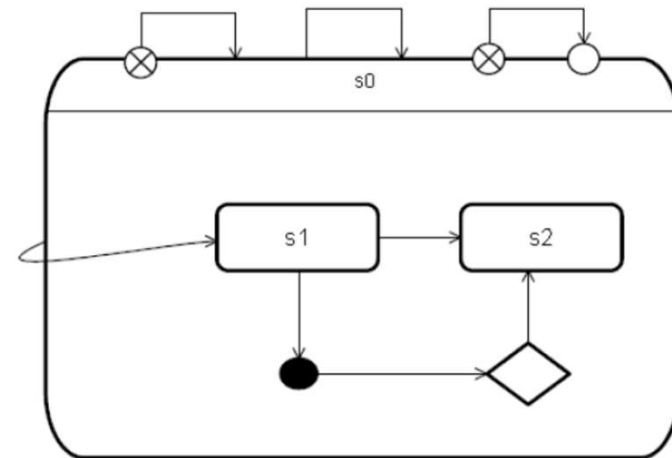


Figure 14.35 External Transitions

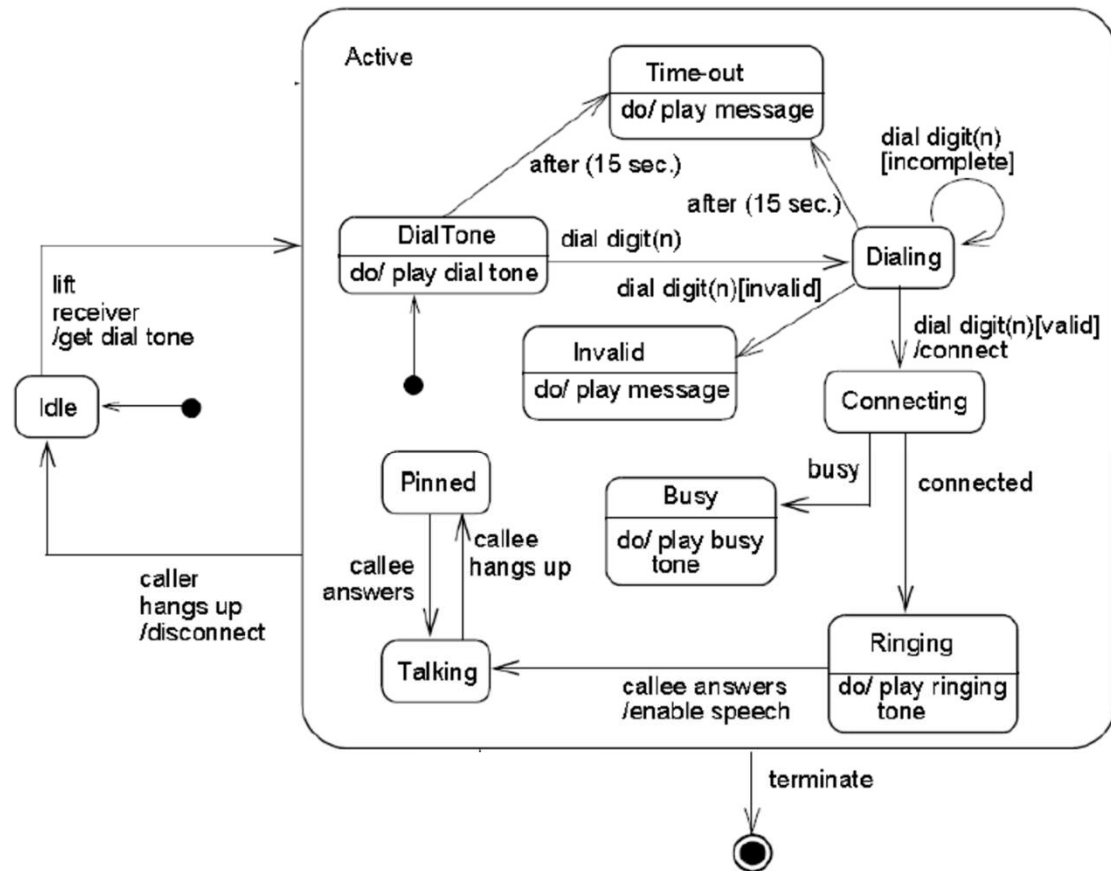
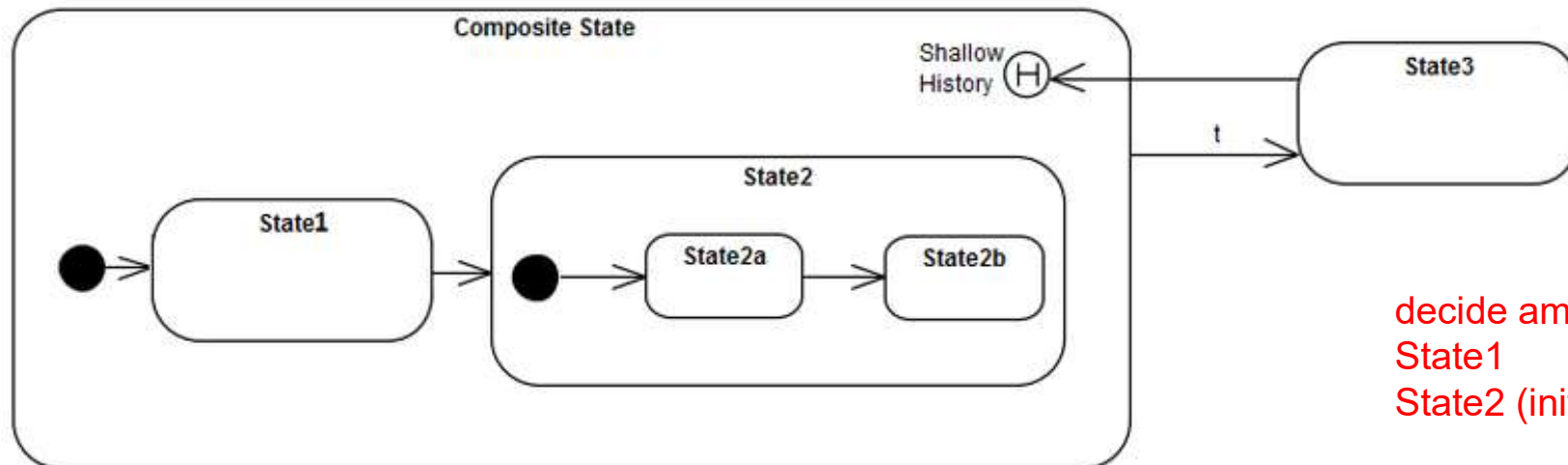


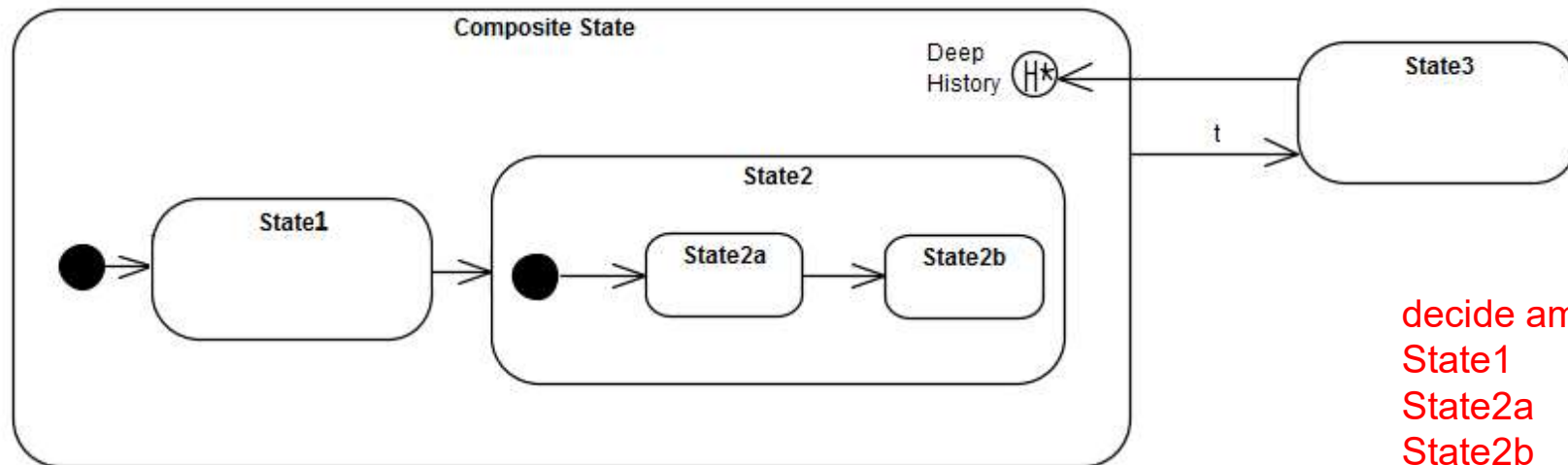
Figure 14.36 StateMachine diagram representing a telephone

SHALLOW HISTORY



decide among:
State1
State2 (initial state)

DEEP HISTORY



decide among:
State1
State2a
State2b

CONCURRENCY IN UML STATECHARTS

AND Superstate

- this superstate is internally divided, by dashed lines, into regions
- each region represents a concurrent FSM
- thus, there are several (one by region) concurrent active states

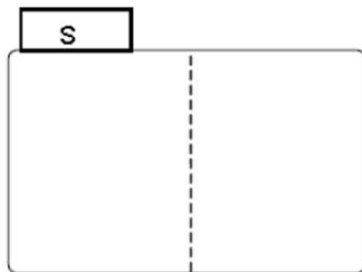


Figure 14.3 Notation for a composite State with Regions

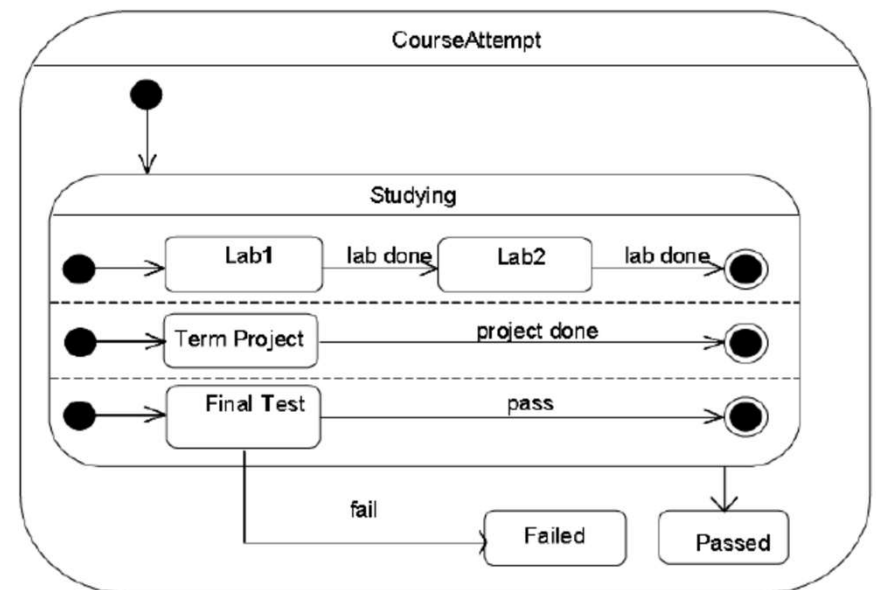


Figure 14.9 Composite State with Regions

GRAPHICAL NOTATION

Region 1 Region 2	UMLShape (Region) - UMLNameLabel (Region) UMLShape (Region) - UMLNameLabel (Region)
--------------------------------	--

CONCURRENCY IN STATECHARTS - EXAMPLES

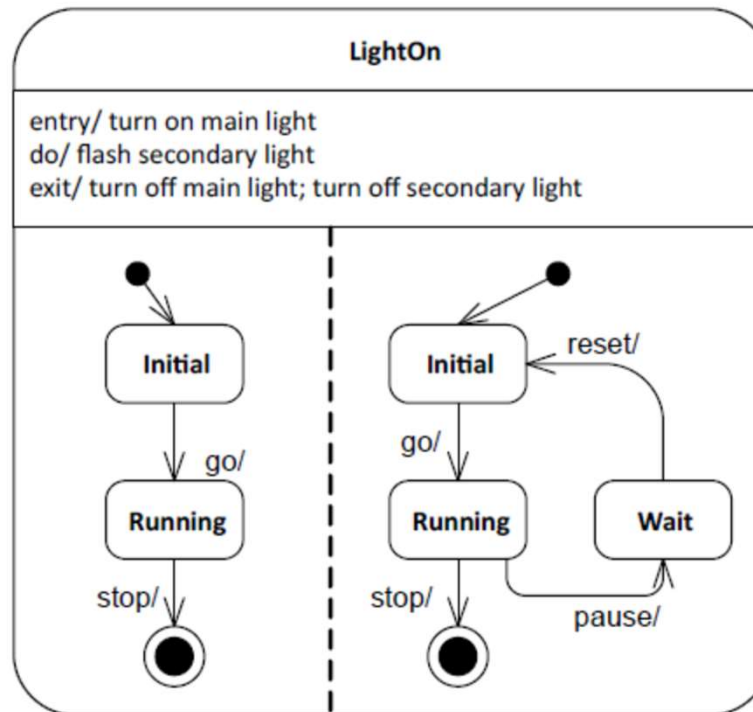


Figure 14.10 Composite State with two Regions and entry, exit, and do Behaviors

CONCURRENCY IN STATECHARTS - EXAMPLES

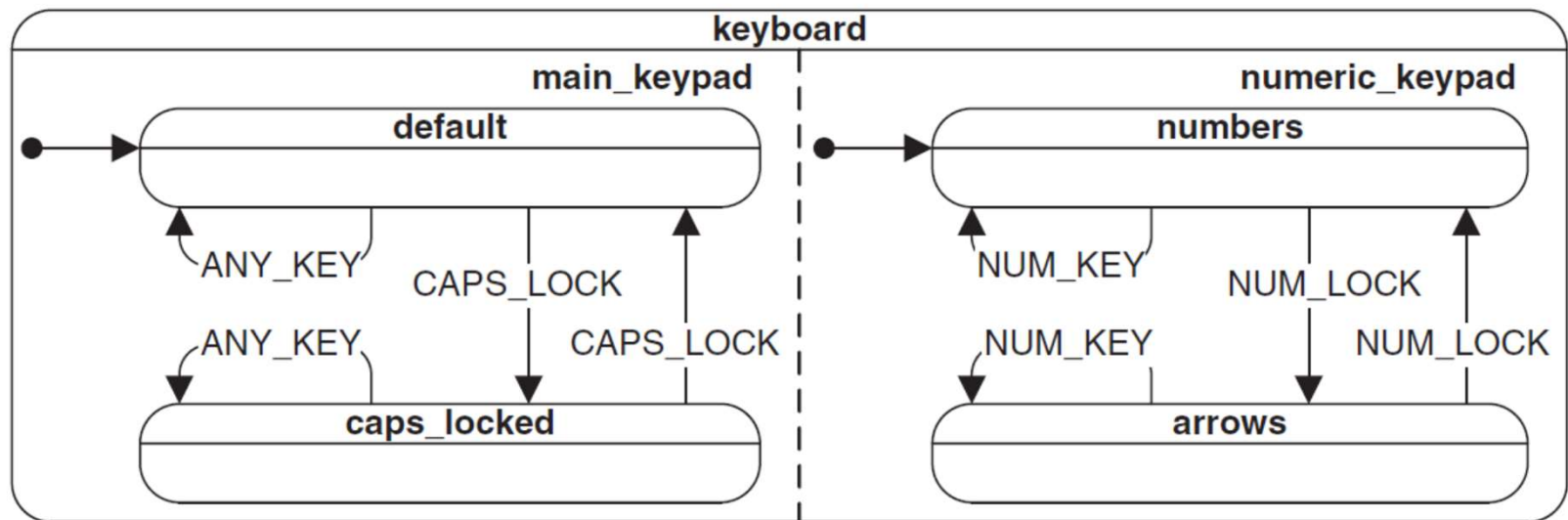
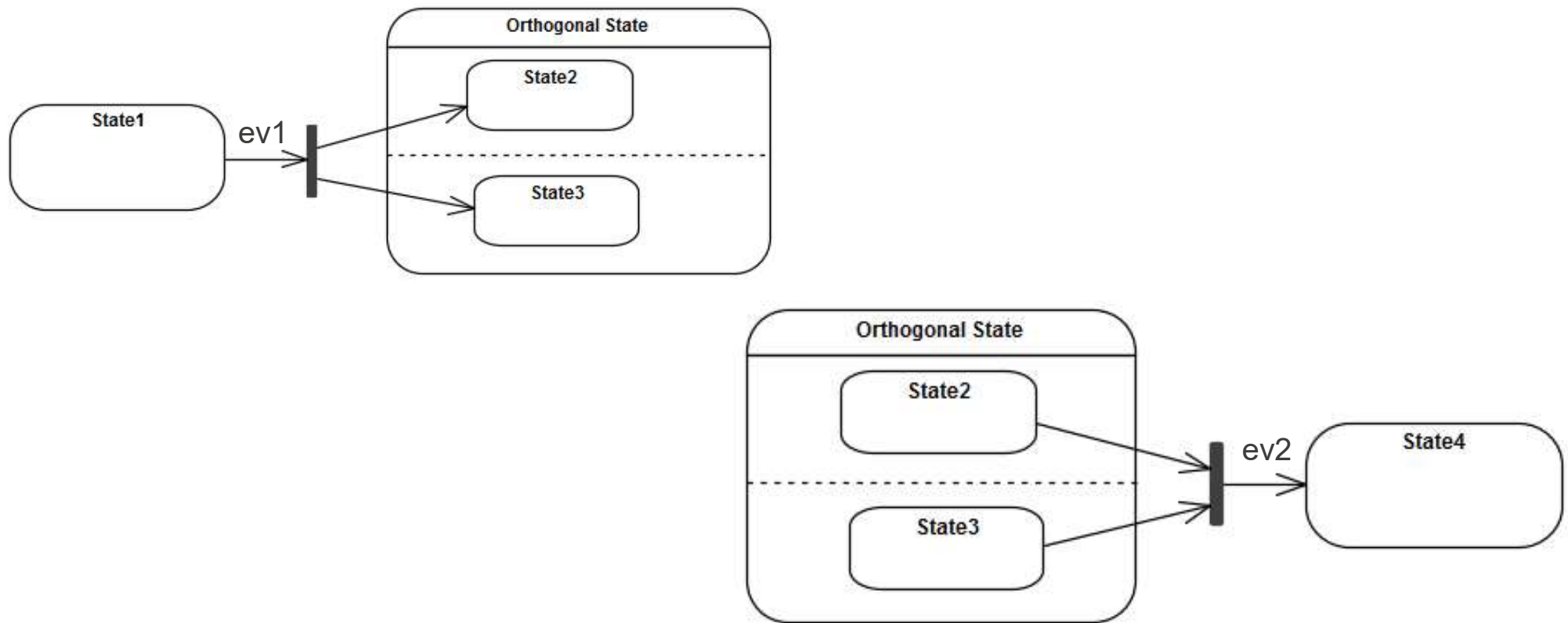
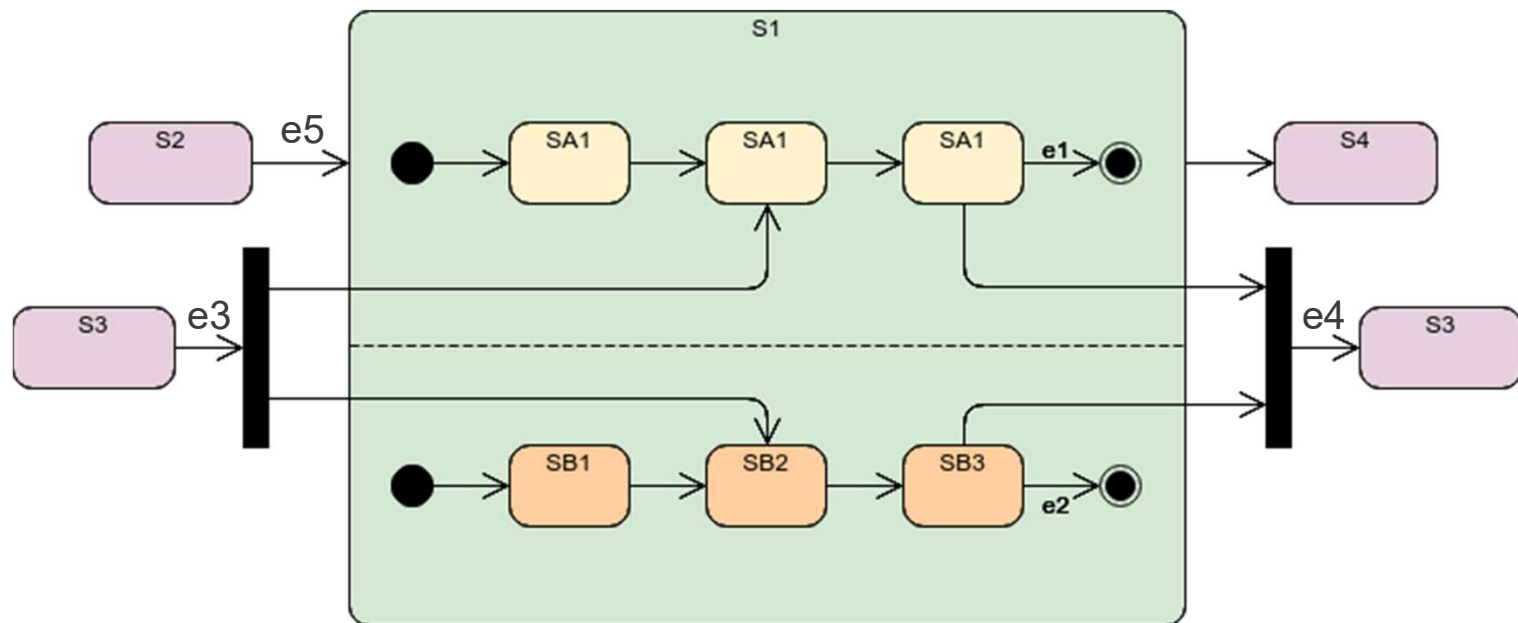


Figure 2.6: Two orthogonal regions (main keypad and numeric keypad) of a computer keyboard.

FORK AND JOIN



CONCURRENCY IN STATECHARTS - EXAMPLES



CODING A FSM

1. Selection by State
2. Selection by Event
3. State x Event Matrix

CONSIDERATIONS

- `state` is the name of a variable that holds the current state in an OR superstate
- External events are detected by HW/ISR and informed to the code via a RTOS mechanism such as message or event

SELECTION BY STATE

```
enum States {B11, B12, B21, B22, B31};
void Th1 ( Card32 p)
{
    PutMsgStr pm;
    States state = B11;

    while (1) {
        Receive(&pm,sizeof(pm));
        switch (state) {
            case B11:
                if( /* event X */) {
                    // actions and state change
                }
                break;
            case B12:
                if( /* event Y */) {
                    // actions and state change
                }
            }
        }
    }
}
```

SELECTION BY EVENT

```
enum States {B11, B12, B21, B22, B31};
void Th1 ( Card32 p)
{
    PutMsgStr pm;
    States state = B11;

    while (1) {
        Receive(&pm,sizeof(pm));    // get event
        int event = pm.command
        switch (event) {
            case XX:
                if( state == B12 ) {
                    // actions and state change
                }
                break;
            case YY:
                if( state == B31) {
                    // actions and state change
                }
            }
        }
    }
}
```

STATE X EVENT MATRIX

FSM is represented by a matrix where each element is a function pointer

`States (*mee[N][M])(States) =`

Each transition has a corresponding function

`{ {f1, f2, f3}, {f1, f5, f4}, ... };`

for each received event a function call is made:

`state = mee[state][event](state);`